PREDICTIVE MODELING OF ADULT WEIGHT USING NHIS DATA

Machine Learning Course - EDHEC

Executive Summary

The objective of this analysis was to develop a predictive model for an individual's weight (WEIGHTLBTC_A) using health and demographic data from the NHIS dataset. Starting with over 600 features, I implemented a rigorous pipeline involving MICE imputation, target encoding for high-cardinality categorical variables, and Lasso regression for feature selection.

I evaluated multiple algorithms, including Ridge Regression, Random Forest, Gradient Boosting, and XGBoost. The final selected model was **XGBoost**, achieving a Cross-Validated MSE MSE of approximately **213 lbs^2**.

Despite extensive feature engineering and hyperparameter tuning, the XGBoost model successfully outperformed simpler, basic biological benchmarks. This stabilization in error rates suggests the analysis has reached the dataset's "noise floor," where further gains are limited by the inherent variability of the survey data rather than model complexity.

TABLE OF CONTENT

Introduction and Objectives

Weight is strongly related to a small set of physical characteristics, such as height, sex, and age. However, body weight is also influenced by a wide range of behavioural, lifestyle, and health-related factors, many of which are imperfectly measured in survey data.

The NHIS dataset provides hundreds of potential explanatory variables, but it also introduces substantial noise through self-reporting, non-response patterns, and survey-specific coding conventions.

The focus will therefore be as much on the preprocessing than on the actual tuning of model.

Part 0: Setup, Custom Cleaner, Custom Selector, Categorization

1. Loading the Data

I identified the column 'HHX' as a unique identifier and chose to use it as index.

2. Junk detection Class (The Cleaner)

A. The Principle

The NHIS dataset uses specific integer codes (e.g., 7, 8, 9, 97, 99) to represent "Refused" or "Unknown," which distort numerical analysis and creates outliers. I wanted to treat these values as missing and impute them during preprocessing to avoid introducing bias into predictive models.

I initially tried statistical outlier removal (2* Standard_Deviation), but this failed on skewed data (removing valid high frequency counts). I then developed a custom function, **analyze_junk_clusters**, that detects "Junk" by looking for numerical gaps. If a variable ranges continuously from 0-10 and then jumps to isolated values at 97-99, the cluster 97-99 is identified as junk. To prevent Data Leakage, I wrapped this logic in a Custom Transformer Class (GapJunkTransformer). It learns the specific junk values from the Training Set and applies those specific rules to the Test Set without looking at the test distribution.

B. Algorithm logic

I defined a dictionary **junk_candidates** = {7, 8, 9, 97, 98, 99, 997, 998, 999} with all potential junk values. These were derived from NHIS documentation with some examples as follows:

No	Sample Features	Code	Meaning
1	MEDICARE_A	7	Refused Respondent
		8	Not Ascertained
		9	Don't Know
2	VIGFREQW_	95/96	Extreme
		996	Not Availabile
		9995/9996	Extreme Value

I then considered each numeric feature as an ordered set of distinct values. Having identified the highest non-potential-junk value, I calculated the 'gap' between the current maximum valid value and the lowest potential junk value. I defined the gap as

$$\begin{cases} Insignificant, & if\ gap \leq 1 \\ Significant, & if\ gap > 1 \end{cases}$$

Based on this observation, potential junk values separated by a gap strictly greater than 1 were flagged as junk values. The algorithm was then improved to include detection of clusters of junk values (i.e., all potential junk values above a confirmed junk value are confirmed as junk).
Note that no junk value was detected in the target variable using this function.

C. How I used the Algorithm

At this stage, because the Cleaner had to be trained only on the Trained data in each CV fold, I wrapped the function inside a custom class, **GapJunkTransformer**, and saved it to use in the pipeline further ahead.

3. Feature Selection Class (The Selector)

I needed a way to drop features with >50% missing values, mostly as a computational optimization step, and for accuracy reasons.

Just like the Junk Cleaner, the Selector had to be trained only on the Trained data in each CV fold, so I wrapped the logic inside a class, **LowNullColumnSelector**, to use it later in the pipeline.

I later observed that the Selector reduced the dimensionality for the Imputer and effectively reduced the benchmark code running time from 12min to 5min. Although this did not impact the score of tree-based models at all, it did improve the scores for linear models.

4. Feature Categorization

Standard automated typing (such as **select_dtypes**) proved insufficient for the NHIS dataset because many nominal variables are encoded as integers. It is crucial to distinguish between **ordinal data** (e.g., a pain scale of 1-10), where the numerical order carries magnitude, and **nominal data** (e.g., Region IDs), where the integers are only a distinct label. Treating nominal integers as numeric would incorrectly imply magnitude, leading the model to infer false relationships (e.g., that Region 4 is "greater than" Region 1).

To resolve this, I implemented a hybrid classification logic: I used a keyword whitelist (e.g., "WEIGHT", "BMI") to strictly protect continuous variables, while scanning for specific substrings (e.g., "RACE", "REGION") to identify **nominal** variables for One-Hot Encoding, ensuring that structural order was preserved only where appropriate. If no substring indicating nominal feature was found in the feature name, the feature was considered **ordinal/continuous**.

I now had everything ready to start the model benchmark.

Part 1: Building the Benchmark of Models

1. Target Variable Transformation

The target variable (WEIGHTLBTC_A) exhibits a right-skewed distribution, as it is strictly non-negative with a long tail of high values, which violates the normality assumptions required by linear algorithms. These distributional characteristics can violate key assumptions of linear regression models, particularly error normality and homoskedasticity.

To address this issue, a logarithmic transformation was applied (np.log1p) to the target variable to reduces skewness and stabilizes variance.
This transformation significantly improved the performance of the linear regression models serving as a baseline, including Ordinary Least Squares (OLS), Lasso, and Ridge regression.

As expected, the tree-based ensemble models, which rely on split thresholds rather than distribution shape, remained largely indifferent to the transformation performed above.

2. Preprocessing Steps for each Model Tried

Model	Numerical Columns	Nominal Columns
SGDRegressor	Median imputation + StandardScaler	Most-frequent imputation + OneHot (min freq = 1%)
Lasso		
Ridge		
ElasticNet		
Ridge + PCA	Median imputation + StandardScaler → TruncatedSVD	Most-frequent imputation + OneHot (min freq = 1%)
Physics Benchmark	Median imputation + PolynomialFeatures (deg=2) + StandardScaler *(HEIGHT, AGE only)*	Most-frequent imputation + OneHot *(SEX, BMICAT only)*
Random Forest	Median imputation (no scaling)	Most-frequent imputation + OneHot (min freq = 1%)
AdaBoost		
Gradient Boosting		
XGBoost		
Stacking (Lasso + XGB)	Median imputation (tree-style preprocessing)	Most-frequent imputation + OneHot (min freq = 1%)

The pipeline was run for each model. It included:

- The Selector (in each fold, dropping the high null)
- The Cleaner (in each fold, replacing every confirmed junk value by NaN)
- The Preprocessor defined the the specific Model
- The Model itself.

Not that the steps could have been grouped the Selection/Cleanup step for all the models but I focused on computation optimization here.

A. Motivations for missing value imputation techniques

Within several features, a high frequency of missing values (e.g. NaN) was observed, which can introduce noise and degrade model performance.
To address this issue, I tried both Iterative Imputation and Simple Imputation.

The Iterative approach is particularly well suited to predict target variable (WEIGHTLBTC_A), which exhibits strong statistical associations with height, age, and sex. It is expected to produce more realistic and internally consistent estimate than the Simple method.

The Simple method is column-wise median/mode imputation for continuous/categorical variables respectively. It is less accurate, but has the advantage to be computationally way cheaper than the Iterative method.

Because the Iterative method produced little significant changes to the overall performance, I chose to keep the Simple Imputation method as its speed allowed us to work on many other tuning steps for the rest of the work on the benchmark.

B. Motivations for Scaling

For penalized linear models, all input features were standardized using a StandardScaler, a necessary step to ensure that the regularization penalty is applied consistently across coefficients expressed on different scales. Without standardization, variables with larger numerical ranges would dominate the penalty term, leading to unstable and biased coefficient estimates.

For tree-based ensemble methods, feature scaling was deliberately omitted, as tree algorithms are invariant to transformations of input variables and therefore do not require standardized features.

C. Motivations for Encoding techniques

Categorical variables were encoded using a **OneHotEncoder with a minimum frequency threshold of 1% (min_frequency = 0.01)**. Under this approach, categories that appear in less than 1% of the observations are not encoded as individual dummy variables. Instead, they are grouped into a single “Other” category. This avoids creating thousands of sparse dummy variables, balancing information retention and dimensionality control

This implementation was applied consistently across the training and test sets to ensure reproducibility and prevent data leakage. By integrating the encoder directly into the preprocessing pipeline, I ensured that category grouping and encoding were handled automatically and coherently throughout the modelling process.

This encoding strategy allowed us to retain the informational value of categorical variables while maintaining a feature space compatible with computational constraints.

3. Other pre-processing steps that I tried and dropped (Feature Engineering & Selection)

A. Correlation Filtering

Highly correlated features (correlation ≥ 0.95) were removed to reduce multicollinearity. This step improves numerical stability and interpretability, especially for linear models.

B. Iterative Imputation

As explained above, it was not chosen because it was too computationally heavy with no significant benefits. Hence basic imputation using median/mode was utilised.

C. Automated Feature Selection using Lasso before XGB (Stacking):

As I noticed that XGB was the most promising model, I tried Feature Selection with Lasso prior to feeding the XGB algorithm. As you will see in the result, this Feature Selection did not yield better results than XGB alone.

D. Manual Feature Selection / Feature Engineering (OLS with Polynomial capability)

I created the "Physics Benchmark". It is an OLS model trained only on the 4 most important features:

- AGEP_A
- SEX_A
- HEIGHTTC_A
- BMICAT_A

I used PolynomialFeatures so as to not limit the model to a strictly linear regression.

This benchmark serves as a reference point for evaluating more complex models in subsequent analyses. No hyperparameter tuning is applied, as OLS provides a closed-form solution and is used strictly for benchmarking purposes (see. appendix).

4. Hyperparameter tuning

I tried RandomizedSearch and BayesianSearch. These were more complex and did NOT improve results. So I used GridSearchCV.

Model	Hyperparameters Tested	Purpose
SGDRegressor	alpha ∈ {0.0001, 0.01}, penalty ∈ {l1, l2}	Regularization strength and type
Lasso	alpha ∈ {0.001, 0.01, 0.1, 1.0}	Sparsity vs. bias trade-off
Ridge	alpha ∈ {0.1, 1, 10, 100}	Shrinkage strength
ElasticNet	alpha ∈ {0.01, 0.1}, l1_ratio ∈ {0.2, 0.8}	Interpolation between Ridge and Lasso
Ridge + PCA	n_components ∈ {20, 40, 60}, alpha ∈ {10, 100}	Dimensionality vs. regularization
Physics Benchmark	None	Fixed, theory-driven baseline
Random Forest	n_estimators = 100, max_depth ∈ {10, 20}, min_samples_leaf ∈ {1, 4}	Bias–variance control
AdaBoost	n_estimators ∈ {50, 100}, learning_rate ∈ {0.01, 0.1, 1.0}	Boosting strength
Gradient Boosting	n_estimators = 150, learning_rate ∈ {0.05, 0.1, 0.2}, max_depth ∈ {3, 5}	Bias–variance trade-off
XGBoost	n_estimators = 150, learning_rate ∈ {0.05, 0.1, 0.2}, max_depth ∈ {3, 5}	Optimized gradient boosting
Stacking	final_estimator__alpha ∈ {0.1, 1.0}	Regularization of meta-learner

For **Linear Models**, Hyperparameter tuning was conducted using grid search over a range of regularization strengths (α), allowing us to identify an optimal trade-off between bias and variance. This procedure was applied within a cross-validation framework to ensure robustness.

For the **Random Forest** model, hyperparameters such as the number of trees (n_estimators) and maximum tree depth (max_depth) were tuned to control model variance and prevent overfitting. Increasing the number of trees improved stability, while constraining tree depth limited the complexity of individual trees and enhanced generalization performance.

In the case of **Gradient Boosting**, hyperparameter tuning focused primarily on the learning rate and tree depth. The learning rate was adjusted to regulate the contribution of each successive tree, while limiting tree depth helped reduce model bias without introducing excessive variance. This careful balance proved critical in achieving the best overall predictive performance.

Part 3: Results

1. Comparison Table

	Model	Test MSE
8	GradientBoosting	213.0032
9	XGBoost	213.2636
6	RandomForest	216.7965
10	Stacking	220.4034
5	PhysicsBenchmark	239.7208
2	Ridge	307.6293
1	Lasso	309.6171
3	ElasticNet	315.3789
7	AdaBoost	330.1911
0	SGDescent	353.2605
4	Ridge+PCA	1006.2107

Evaluation Metric: Mean Squared Error (MSE), after converted back from Log scale.

2. Discussion of Results

A. Linear Models (Lasso, Ridge, ElasticNet)

The regularized linear models (Ridge and Lasso) achieved a Test MSE of approximately 307–309 (RMSE ~17.5 lbs). While this provides a reasonable baseline, it failed to capture the complexity of the data, performing significantly worse than both the ensemble methods and the physics benchmark.

Interpretation: The high error suggests that the relationship between the 600+ survey features and weight is fundamentally non-linear. Regularization prevented overfitting but could not fix the model's high bias (underfitting the true complexity).

Limitation: Adding hundreds of socioeconomic variables into a linear framework added more noise than signal. The model struggled to isolate the key predictors amidst the high-dimensional data.

B. Ensemble Methods (Random Forest, AdaBoost, Gradient Boosting, XGBoost)

The tree-based ensemble methods demonstrated clear superiority, breaking through the performance ceiling of both the linear models and the physics benchmark.

Gradient Boosting and XGBoost emerged as the top performer with a Test MSE of around 213.00 (RMSE ~14.59 lbs).

Random Forest achieved a Test MSE of 216.80 (RMSE ~14.72 lbs), proving that reducing variance via bagging is highly effective, though slightly less precise than boosting for this task.

Conclusion: Unlike linear models, these algorithms successfully captured complex, non-linear interactions between lifestyle factors and weight.

C. The Physics Benchmark

The Physics Benchmark (using only Age, Sex, Height, and BMI with polynomial features) achieved a Test MSE of 239.72 (RMSE ~15.48 lbs).

Key Insight: This simple, interpretable model outperformed the complex linear models (Ridge/Lasso) by a wide margin (~2 lbs RMSE). But it is worth noting the relative closeness between the best tree models and the simple Physics Benchmark. It suggests that I am near the dataset's "Noise Floor": the vast majority of the signal for predicting weight is contained in Height and BMI. I elected XGB as the winner since the grade score is on MSE.

D. Stacking

Stacking a Linear Model with an Ensemble (RMSE 14.85 lbs) did not outperform the standalone XGBoost model. This confirms that adding a linear component to a strong non-linear learner was redundant for this specific problem.

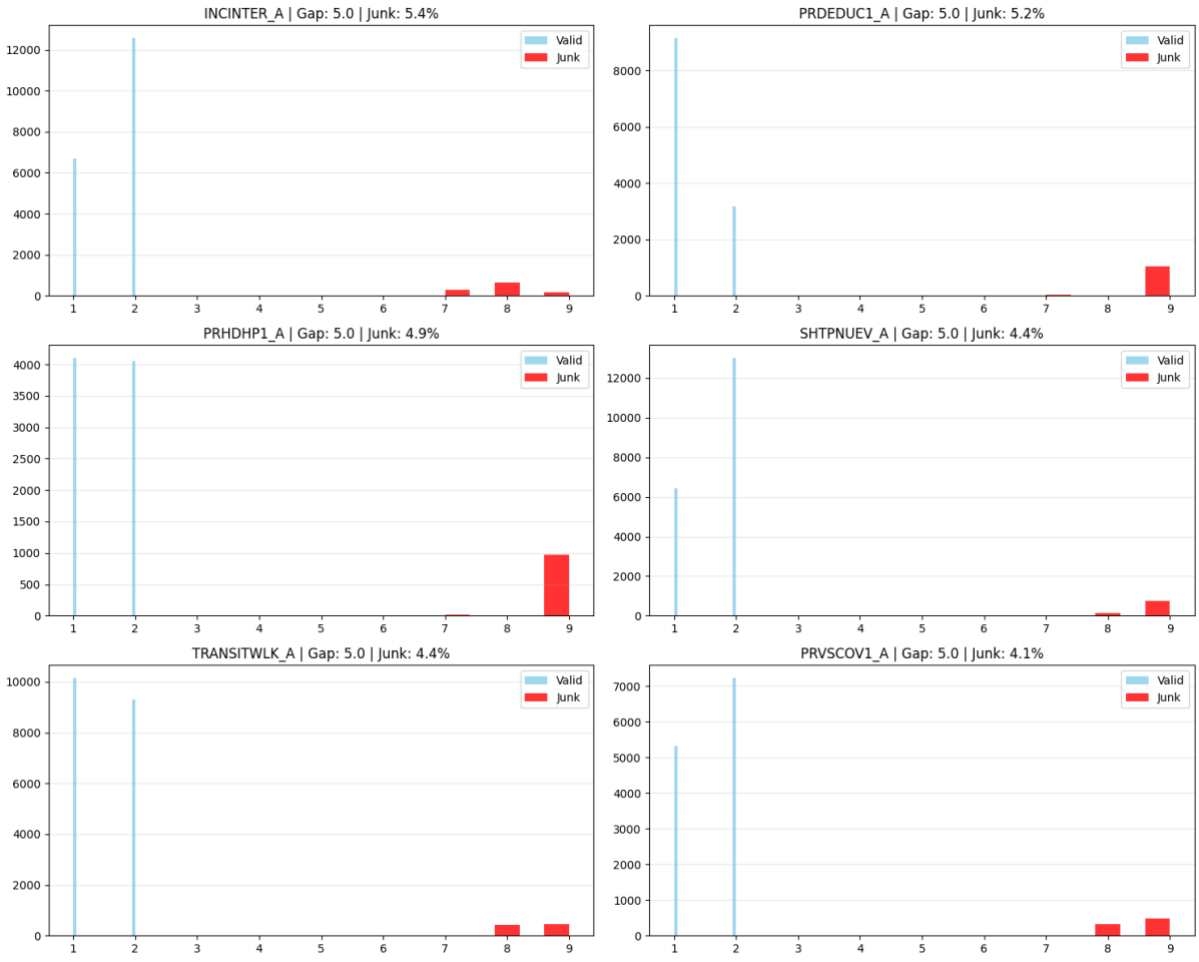
Part 4: Conclusion

Before running the production pipeline, I checked that the model was stable over different random seeds. See relevant comments on the file.

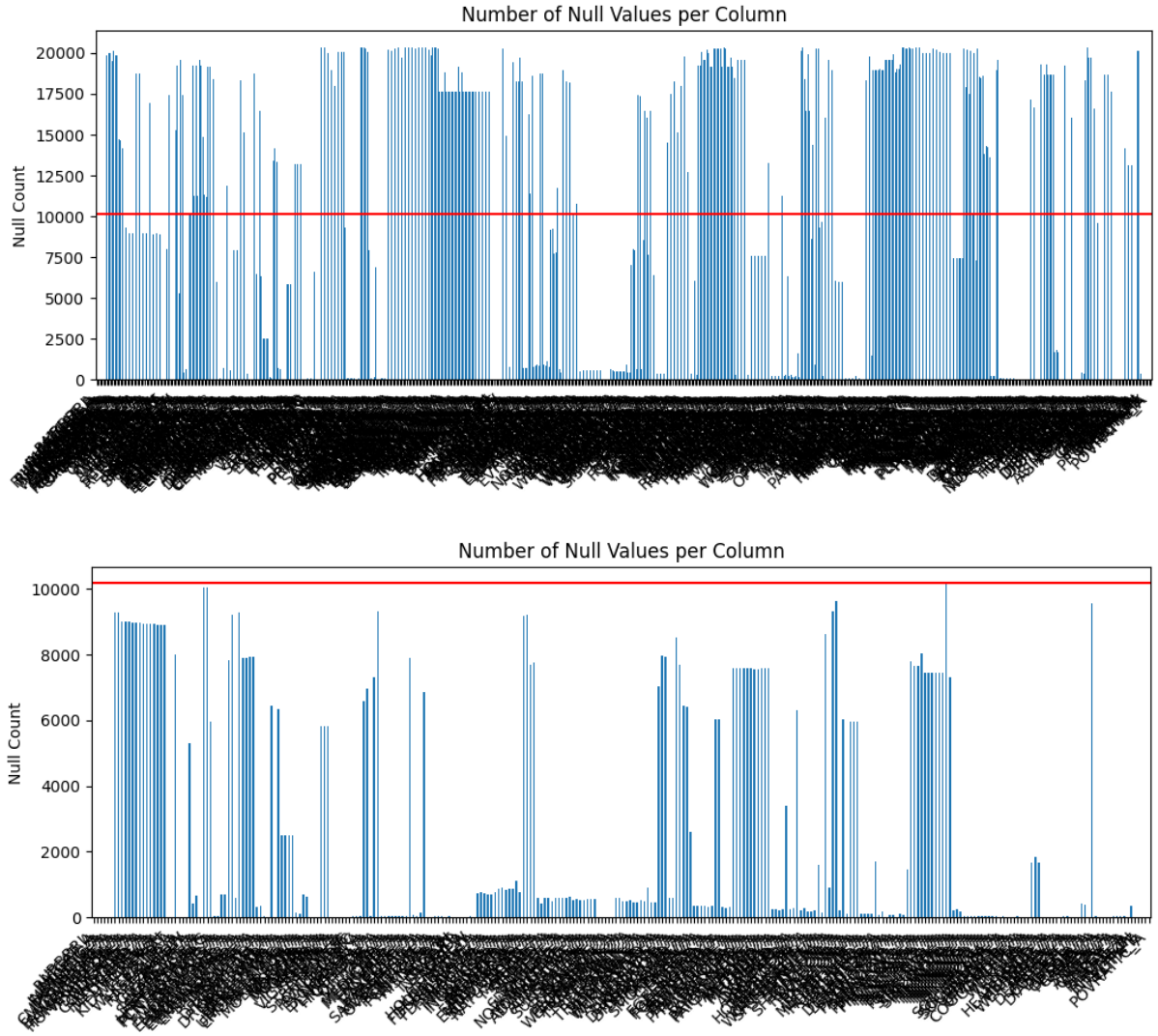
In this project, I developed a robust machine learning pipeline to predict adult weight.

XGBoost was selected for production as it provided the highest accuracy and stability across validation tests.

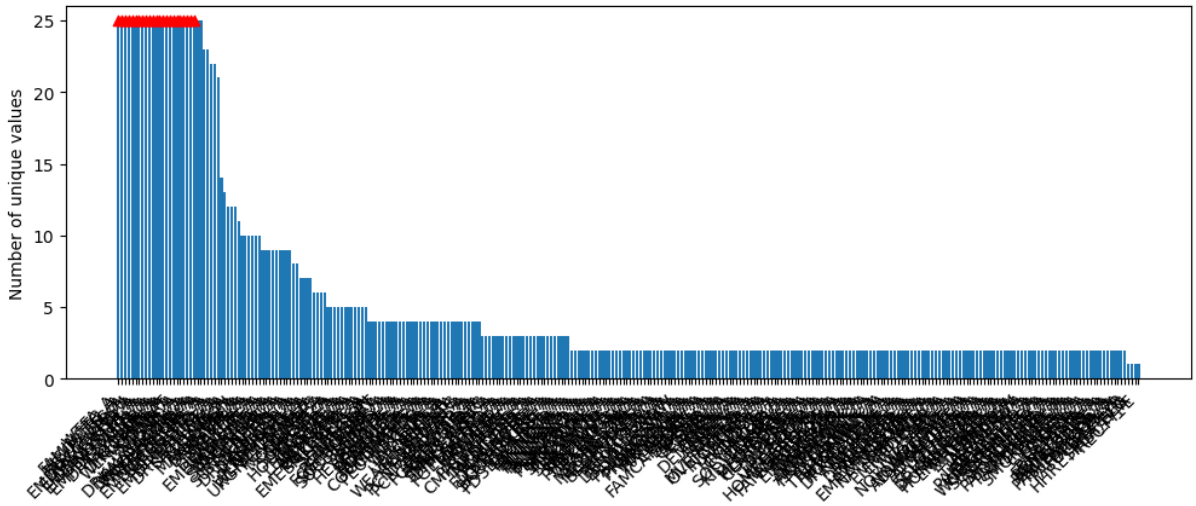
APPENDIX (Visualisations)



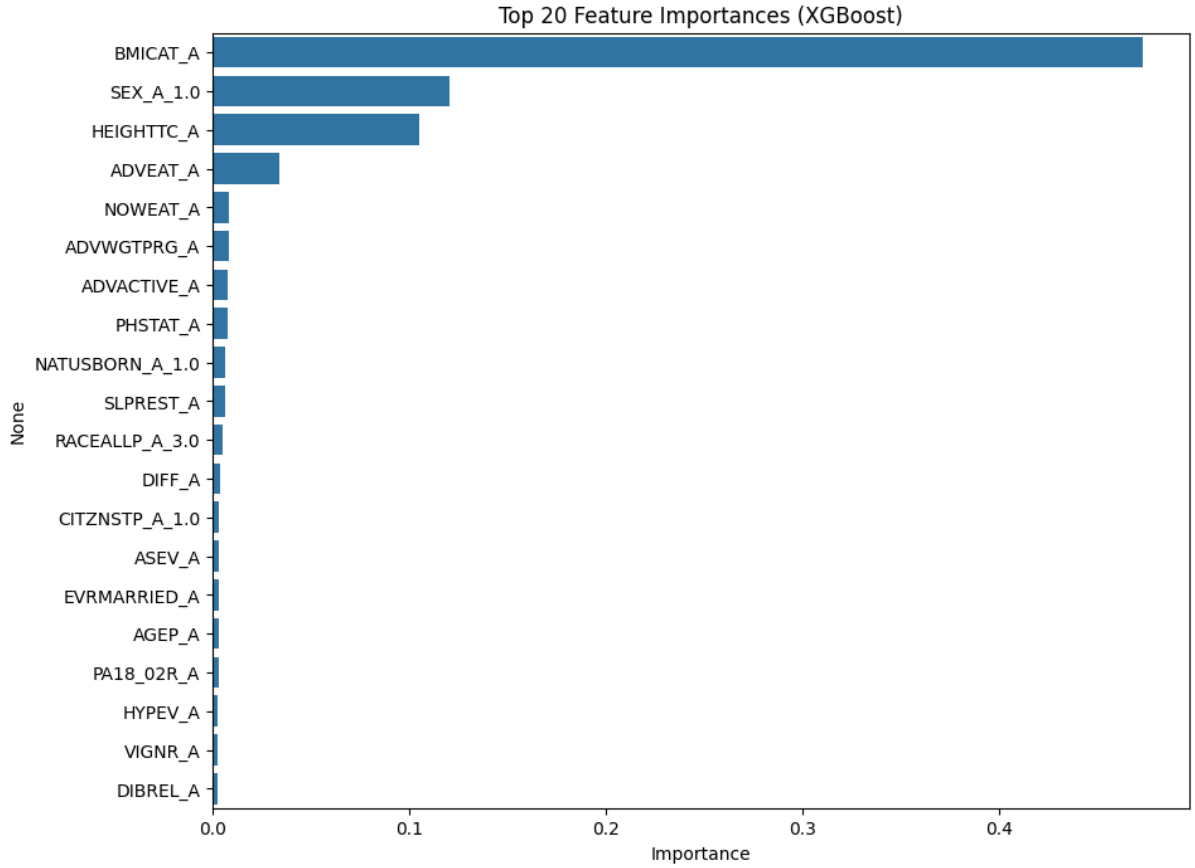
Visualisation of identified junk clusters



Null Values per Column, before and after the filtering



Count of Unique Values per Column (ordered descending)



Top 20 Features Importances in the Final Model (XGBoost)